

EXPERIMENT-4

Build an Artificial Neural Network using Backpropagation.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
from google.colab import files

print("="*70)
print("  ARTIFICIAL NEURAL NETWORK")
print("="*70)

uploaded = files.upload()

data = pd.read_csv("Iris.csv")

print("\nDataset\n")
print(data.head(10))

X = data.iloc[:,1:5]
y = data.iloc[:,5]

encoder = LabelEncoder()
y = encoder.fit_transform(y)
```

```

X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.3,random_state=42)

model = MLPClassifier(hidden_layer_sizes=(5,),
                      max_iter=1000,
                      random_state=1)

print("\nTraining Neural Network...")

model.fit(X_train,y_train)

prediction=model.predict(X_test)
accuracy=accuracy_score(y_test,prediction)

print("\nTraining Completed Successfully")
print("\nActual Values")
print(y_test[:20])
print("\nPredicted Values")
print(prediction[:20])
print("\nAccuracy")
print(round(accuracy*100,2),"%")
print("\nNumber of Layers :",model.n_layers_)
print("Number of Outputs :",model.n_outputs_)
print("Activation Function :",model.activation)

```

Sample Input:

```

...  =====
      ARTIFICIAL NEURAL NETWORK
      =====
      Choose Files Iris.csv
      Iris.csv(text/csv) - 5107 bytes, last modified: 8/5/2026 - 100% done
      Saving Iris.csv to Iris.csv

```

Sample Output:

Dataset

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
5	6	5.4	3.9	1.7	0.4	Iris-setosa
6	7	4.6	3.4	1.4	0.3	Iris-setosa
7	8	5.0	3.4	1.5	0.2	Iris-setosa
8	9	4.4	2.9	1.4	0.2	Iris-setosa
9	10	4.9	3.1	1.5	0.1	Iris-setosa

Training Neural Network...

Training Completed Successfully

Actual Values

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2]

Predicted Values

[1 0 2 2 1 0 1 2 1 1 2 0 0 0 0 2 2 1 2 2]

Accuracy

91.11 %

Number of Layers : 3

Number of Outputs : 3

Activation Function : relu